

1. What is the difference between latency and throughput in model inference, and which factor influences model performance more and how?

Latency is the time taken by a model to process a single request and produce an output. It is usually measured in milliseconds (ms) or seconds. Lower latency means the model responds faster.

Throughput is the amount of work a model can process in a given period of time. In model inference, it can be measured as requests per second or tokens per second. Higher throughput means the system can handle more requests in the same amount of time.

For example, a chatbot used by a single user may prioritize low latency, while a company serving thousands of users may prioritize high throughput.

Both affect model performance, but which one matters more depends on the application. Interactive applications generally focus more on latency, whereas large-scale serving systems often focus more on throughput.

2. What is KV Cache and how does it help with model generation?

KV Cache (Key-Value Cache) is a technique used by Transformer-based models to make text generation faster. When a model generates text, it processes the input tokens one by one. For each token, the model calculates Key (K) and Value (V) information during the attention process.

Normally, when generating a new token, the model would calculate the Key and Value information for all the previous tokens again. This would waste computation, especially when the input or generated text becomes long.

With KV Cache, the Key and Value information of previously processed tokens is stored in memory.

3. What is quantization, and what is the trade-off between INT8/INT4 quantization and model accuracy? Also give an example with calculations and formula of how quantization can be performed.

Quantization is the process of converting the numbers used by a model from a higher precision format to a lower precision format. For example, a model can use FP32 numbers and convert them to INT8 or INT4 numbers.

The main purpose of quantization is to reduce the model's memory size and make inference faster. Since INT8 and INT4 use fewer bits than FP32, they require less memory to store and move.

However, there is a trade-off with accuracy. When the original values are converted to lower precision, some information is lost. Therefore, the model may have a small drop in accuracy. In general, INT8 usually preserves accuracy better than INT4, while INT4 provides greater memory savings.

Example of Quantization

Consider the following FP32 values:

$$[-2.0, -1.0, 0.0, 1.0, 2.0]$$

For INT8, the quantization range is:

$$[-128, 127]$$

The scale can be calculated using:

$$\text{Scale} = \frac{\text{Max} - \text{Min}}{Q_{\text{max}} - Q_{\text{min}}}$$

Substituting the values:

$$\text{Scale} = \frac{2 - (-2)}{127 - (-128)}$$

$$\text{Scale} = \frac{4}{255} \approx 0.0157$$

The zero-point is calculated as:

$$\text{ZeroPoint} = \text{round} \left(Q_{\text{min}} - \frac{\text{Min}}{\text{Scale}} \right)$$

Therefore:

$$\text{ZeroPoint} = \text{round} \left(-128 - \frac{-2}{0.0157} \right) \approx 0$$

The quantized value can then be calculated using:

$$q = \text{round} \left(\frac{x}{\text{Scale}} + \text{ZeroPoint} \right)$$

For example, for $x = 1.0$:

$$q = \text{round} \left(\frac{1.0}{0.0157} + 0 \right)$$

$$q \approx 64$$

Therefore, the FP32 value 1.0 is represented by approximately 64 in INT8.

4. Name one real inference serving framework and briefly describe one optimization it does.

vLLM is a framework used to efficiently serve and run large language models for users and applications. It focuses on improving the speed and efficiency of LLM inference.

One important optimization used by vLLM is PagedAttention. It manages the KV Cache more efficiently by dividing it into smaller blocks instead of requiring one large continuous memory space.

This helps reduce wasted GPU memory and allows the system to handle more requests at the same time.

In simple terms, PagedAttention helps vLLM use KV Cache memory more efficiently, which improves memory utilization and overall inference throughput.